

پروژه درس یادگیری عمیق ۱۴۰۴

هدف: طراحی سامانه‌ی پایش وضعیت راننده با استفاده از vision transformer

شیما ضرغامی

(۱) در ابتدا مجموعه داده‌ی مناسب این پروژه را باید پیدا میکردیم. از مجموعه داده‌های گفته شده، میخواستم state farm را دانلود کنم ولی باید اکانت Kaggle را verify میکردیم ولی شماره‌ی ایران را قبول نکرد و با شماره مجازی هم نشد. بجای آن من دیتاست AUC را از Kaggle دانلود کردم که دقیقاً ساختار state farm را داشت ولی در حجم کمتر. اما متأسفانه پسوورد میخواست و پسووردش در هیچ جا گفته نشده بود. در آخر یک دیتاست مشابه دیگر پیدا کردم. این دیتاست ساختار مشابه‌ای با state farm دارد و در لینک زیر قابل مشاهده است.

<https://github.com/AmalEzzouhri/Driver-Distraction-Dataset?tab=readme-ov-file>

ساختار داده: این داده ۱۰ کلاس مختلف دارد که به شرح زیر است:

- C0: Safe driving
- C1: Texting - right hand
- C2: Talking on the phone - right hand
- C3: Texting - left hand
- C4: Talking on the phone - left hand
- C5: Operating the radio
- C6: Drinking
- C7: Reaching behind
- C8: Tidying up hair or applying makeup
- C9: Talking to passenger.

(۲) پیش پردازش داده‌ها

الف) تبدیل‌های اعمال شده روی تصاویر

```
train_transform = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.RandomHorizontalFlip(p=0.3),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

- **Resize**: تغییر اندازه همه تصاویر به 224×224 . می‌دانیم مدل ViT به اندازه ثابت نیاز دارد و همینطور معمولاً از patch size ۱۶ استفاده می‌شود، با ورودی ۲۲۴ در ۲۲۴ تعداد patch برابر ۱۹۶ می‌شود که از نظر محاسباتی و حافظه بهینه است.
- **RandomHorizontalFlip**: برگرداندن تصادفی افقی که باعث افزایش داده و جلوگیری از overfitting می‌شود. در اینجا من احتمال ۰.۳ را در نظر گرفتم.
- **ColorJitter**: تغییر تصادفی روشنایی و کنتراست که باعث می‌شود مدل به تغییرات نور و شرایط تصویربرداری مقاوم شود و از وابستگی به رنگ و روشنایی خاص جلوگیری شود.
- **ToTensor**: به تانسور تبدیل می‌کند.
- **Normalize**: نرمال سازی انجام می‌دهد.

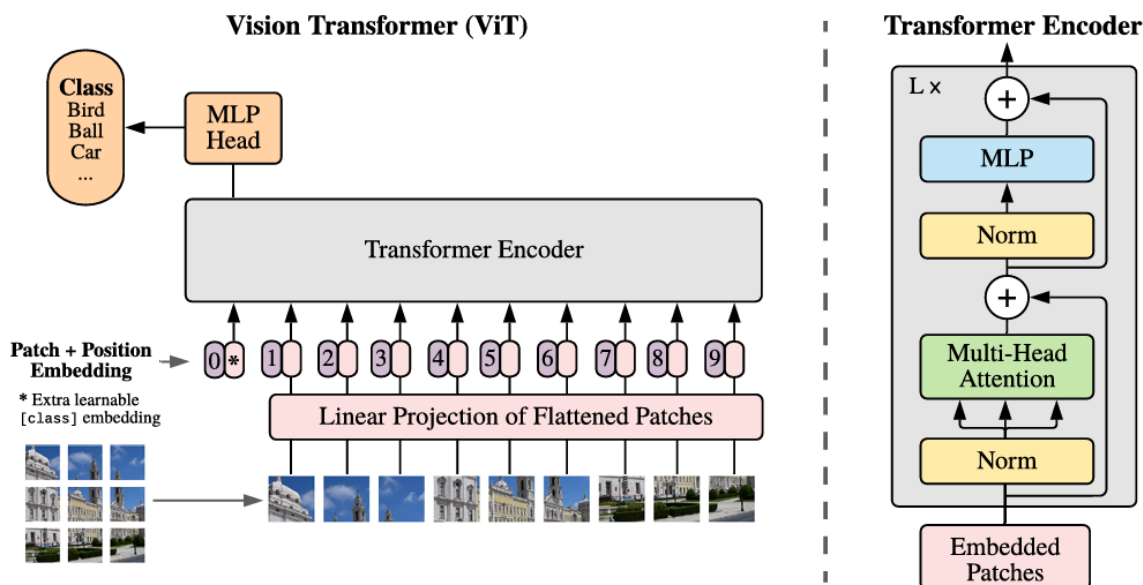
(۳) Vision Transformer (ViT) چیست؟

Vision Transformer (ViT) مدلی برای پردازش تصویر است که ایده‌ی اصلی آن از معماری Transformer در NLP گرفته شده است. یک معماری نوین برای پردازش تصاویر است که از مفهوم "توجه" (Attention) استفاده می‌کند. در ViT، تصویر مانند یک جمله در نظر گرفته می‌شود و به جای استفاده از کانولوشن (CNN)، از مکانیزم Self-Attention برای استخراج ویژگی‌ها استفاده می‌شود.

ایده اصلی ViT

در مدل‌های CNN تصویر به صورت محلی پردازش می‌شود و فیلترها فقط همسایه‌های نزدیک را می‌بینند. به همین صورت وابستگی‌های دور به صورت تدریجی یاد گرفته می‌شوند اما در Vision Transformer تصاویر به قطعات کوچک (patch) تقسیم می‌شود و هر patch مثل یک "کلمه" در جمله در نظر گرفته می‌شود. تمام patch‌ها به صورت همزمان با هم ارتباط برقرار می‌کنند که یعنی مدل از همان ابتدا کل تصویر را به صورت

سراسری تحلیل می‌کند.



این تصویر ساختار کلی ViT را نشان می‌دهد. مرحله به مرحله آنرا بررسی می‌کنیم:

- **Input Image** تصویر ورودی: تصویر ورودی معمولاً ابعاد $H * W * C$ را دارد که برای مثال $224 * 224 * 3$ در ViT می‌باشد که H ارتفاع، W عرض و C تعداد کانال (RGB) است. در ViT تصویر مستقیماً وارد شبکه نمی‌شود، بلکه ابتدا به قطعات کوچک تقسیم می‌شود.
- **Patch Extraction** (تقسیم به پچ): تصویر به بلوک‌های ثابت با اندازه P در P تقسیم می‌شود. اگر: $P = 16$ و تصویر 224×224 باشد:

$$\frac{224}{16} = 14$$

پس تعداد پچ‌ها:

$$N = 14 * 14 = 196$$

هر پچ یک ناحیه محلی از تصویر است.

- **Flattened Patches** (تخت‌سازی پچ‌ها) هر پچ

$$16 \times 16 \times 3$$

به یک بردار یک‌بعدی تبدیل می‌شود:

$$16 \times 16 \times 3 = 768$$

پس هر پیچ به برداری با طول ۷۶۸ تبدیل می‌شود. در نتیجه در این مرحله داریم: 196×768

- Linear Projection (Patch Embedding)

هر بردار ۷۶۸ بعدی وارد یک لایه خطی می‌شود:

$$x_p E$$

که x_p بردار پیچ و E ماتریس وزن قابل یادگیری می‌باشد. هر پیچ حالا یک token embedding است (مثل کلمات در NLP).

- Class Token ([CLS])

یک بردار قابل یادگیری به ابتدای توکن‌ها اضافه می‌شود: x_{class} در نتیجه تعداد توکن‌ها $N + 1 = 197$ می‌شود. این توکن نماینده کل تصویر است و در پایان برای طبقه‌بندی استفاده می‌شود.

- Positional Embedding

چون Transformer ذاتاً ترتیب مکانی را نمی‌فهمد، یک embedding مکانی به هر توکن اضافه می‌شود، این باعث می‌شود مدل بداند هر پیچ در چه موقعیتی قرار دارد.

- Transformer Encoder (بلوک اصلی)

این بلوک L بار تکرار می‌شود.

مثلاً:

- ViT-Base $\rightarrow L=12$

- ViT-Large $\rightarrow L=24$

هر لایه شامل دو زیرلایه اصلی است:

- Multi-Head Self Attention

مرحله ۱: محاسبه Q, K, V

$$X_W Q = Q$$

$$X_W K = K$$

$$X_W V = V$$

مرحله ۲: محاسبه Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

این یعنی هر پیچ با همه پیچها مقایسه می شود.

Multi-Head هم یعنی به جای یک attention، چند attention موازی داریم:

$$head_i = Attention(Q_i, K_i, V_i)$$

سپس:

$$MultiHead = Concat(head_1, \dots, head_h)W_O$$

این اجازه می دهد مدل همزمان چند نوع رابطه را یاد بگیرد.

- Residual Connection

بعد از Attention:

$$X' = X + Attention(X)$$

که این باعث جلوگیری از vanishing gradient و پایداری آموزش می شود.

- Layer Normalization

قبل از هر زیرلایه، نرمال سازی روی بعد ویژگی انجام می شود و باعث پایدار شدن توزیع فعال سازی ها می شود.

- MLP Block

ساختار:

$$MLP(X) = W_2 GELU(W_1 X)$$

Vision Transformer ابتدا تصویر را به پچ‌های ثابت تقسیم کرده و هر پچ را به یک بردار embedding

تبدیل می‌کند. سپس با افزودن توکن کلاس و embedding مکانی، توالی حاصل را به مجموعه‌ای از بلوک‌های Transformer Encoder می‌دهد. هر بلوک شامل Multi-Head Self Attention و یک شبکه MLP با اتصال‌های باقیمانده و نرمال‌سازی است. در نهایت، بردار CLS برای طبقه‌بندی تصویر استفاده می‌شود.

Vision Transformer با تقسیم تصویر به پچ‌های 16×16 و استفاده از مکانیزم Self-Attention، قادر به درک روابط بین اجزای مختلف تصویر (دست‌ها، صورت، اشیاء داخل ماشین) است. این ویژگی برای تشخیص وضعیت راننده بسیار مهم است، زیرا همزمان به چند ناحیه (مثلاً دست و گوشی) توجه می‌کند، روابط فضایی (مثلاً نزدیک بودن دست به دهان برای نوشیدن) را بهتر درک می‌کند و در برابر تغییرات ظاهری (نور، زاویه دوربین) مقاومت می‌کند.

مدل پیاده‌سازی شده با ۴ لایه Transformer و ۶ head attention، تعادل مناسبی بین دقت و سرعت برقرار کرده و برای تشخیص بلادرنگ مناسب است. من دو بهینه‌ساز adam و adamw را استفاده کردم.

۴) دلایل انتخاب پارامترهای Vision Transformer

Image Size = 224: اندازه استاندارد دیتاست ImageNet اکثر معماری‌های مدرن با این سایز تست شده‌اند.

این سایز جزئیات تصویر را حفظ می‌کند، هزینه محاسباتی GPU نرمال است. اگر تصویر بزرگ‌تر شود:

تعداد patch زیاد می‌شود و حافظه به شدت بالا می‌رود (attention quadratic هست)

Patch Size = 16: در مقاله اصلی An Image is Worth 16x16 Words هم ۱۶ استفاده شده است اما

دلیل ریاضیاتی دارد، اگر تصویر ۲۲۴ باشد، $16 / 224 = 14$ پس تعداد patch برابر می‌شود با $14 \times 14 = 196$

این تعداد خوب است زیرا: نه آنقدر زیاد است که attention سنگین شود و نه خیلی کم است که اطلاعات از بین برود.

برای مثال اگر patch=8 بود، تعداد patch میشود ۷۸۴ که در اینصورت محاسبه attention خیلی سنگین میشود.

Embedding dim : Embedding Dimension ظرفیت مدل رو مشخص می‌کند. هر چقدر بزرگ‌تر قدرت نمایش بیشتر می‌شود ولی پارامتر بیشتر و overfitting محتمل‌تر است. من ۵۱۲ انتخاب کردم که نه خیلی زیاد است و نه خیلی کم.

: Number of Heads = 8

در multi-head attention: $\text{head_dim} = \text{embed_dim} / \text{num_heads}$ باید قابل تقسیم باشه.

چند head داشتن باعث می‌شود مدل همزمان چند نوع وابستگی فضایی یاد بگیرد و الگوهای مختلف توجه رو capture

Batch Size = 32 : محدودیت GPU، و تعادل بین نویز گرادین و مصرف حافظه. هرچقدر Batch بزرگ‌تر باشد آنگاه stable gradient و هرچقدر Batch کوچک‌تر باشد generalization بهتر است.

Epoch ۲۰ : چون Loss بعد از حدود ۱۵ اپوک تغییراتش جزئی می‌شود و افزایش بیشتر بهبود جزئی دارد. همچنین جلوگیری از overfitting می‌کند.

هایپرپارامترهای مدل بر اساس محدودیت‌های محاسباتی، اندازه مجموعه داده، و بهترین روش‌های معماری گزارش شده در مقاله اصلی Vision Transformer انتخاب شدند. اندازه پیچ ۱۶ تضمین‌کننده یک توازن مناسب بین قدرت تفکیک‌پذیری مکانی و هزینه محاسباتی است. بعد Embedding و تعداد attention heads به گونه‌ای انتخاب شدند که تعادلی بین ظرفیت نمایشی و مصرف حافظه GPU برقرار کنند. بهینه‌ساز AdamW به دلیل مکانیزم کاهش وزن مستقل (decoupled weight decay) انتخاب شد که تعمیم‌پذیری را در معماری‌های مبتنی بر ترانسفورمر بهبود می‌بخشد. مدل برای ۲۰ (epoch) آموزش داده شد، زیرا عملکرد اعتبارسنجی پس از این نقطه به حالت پایدار (plateau) می‌رسید.

(۵) نتایج اجرای کد:

AdamW	Adam
Epoch 1/20 Train Loss: 2.1589 Acc: 0.2169 Val Loss: 1.4797 Acc: 0.4147	Epoch 1/20 Train Loss: 2.3123 Acc: 0.1884 Val Loss: 2.1391 Acc: 0.2558

Epoch 2/20 Train Loss: 0.6170 Acc: 0.7812 Val Loss: 0.3342 Acc: 0.9031 Epoch 3/20 Train Loss: 0.1437 Acc: 0.9544 Val Loss: 0.1554 Acc: 0.9483 Epoch 4/20 Train Loss: 0.0775 Acc: 0.9758 Val Loss: 0.0336 Acc: 0.9858 Epoch 5/20 Train Loss: 0.1393 Acc: 0.9544 Val Loss: 0.0472 Acc: 0.9845 Epoch 6/20 Train Loss: 0.0265 Acc: 0.9913 Val Loss: 0.0213 Acc: 0.9974 Epoch 7/20 Train Loss: 0.0219 Acc: 0.9952 Val Loss: 0.0157 Acc: 0.9961 Epoch 8/20 Train Loss: 0.0061 Acc: 0.9981 Val Loss: 0.0177 Acc: 0.9974 Epoch 9/20 Train Loss: 0.0084 Acc: 0.9968 Val Loss: 0.0359 Acc: 0.9897 Epoch 10/20 Train Loss: 0.0083 Acc: 0.9981 Val Loss: 0.0933 Acc: 0.9845	Epoch 2/20 Train Loss: 1.8886 Acc: 0.3174 Val Loss: 1.4746 Acc: 0.4406 Epoch 3/20 Train Loss: 0.7956 Acc: 0.7269 Val Loss: 0.4699 Acc: 0.8656 Epoch 4/20 Train Loss: 0.2868 Acc: 0.9205 Val Loss: 0.1620 Acc: 0.9535 Epoch 5/20 Train Loss: 0.1421 Acc: 0.9664 Val Loss: 0.0916 Acc: 0.9845 Epoch 6/20 Train Loss: 0.1661 Acc: 0.9567 Val Loss: 0.1382 Acc: 0.9677 Epoch 7/20 Train Loss: 0.0828 Acc: 0.9796 Val Loss: 0.0489 Acc: 0.9858 Epoch 8/20 Train Loss: 0.0852 Acc: 0.9790 Val Loss: 0.1965 Acc: 0.9522 Epoch 9/20 Train Loss: 0.3503 Acc: 0.8930 Val Loss: 0.0700 Acc: 0.9845 Epoch 10/20 Train Loss: 0.0594 Acc: 0.9900 Val Loss: 0.0456 Acc: 0.9910
--	--

Epoch 11/20 Train Loss: 0.0247 Acc: 0.9906 Val Loss: 0.0070 Acc: 0.9974	Epoch 11/20 Train Loss: 0.0608 Acc: 0.9910 Val Loss: 0.0445 Acc: 0.9935
Epoch 12/20 Train Loss: 0.0008 Acc: 1.0000 Val Loss: 0.0057 Acc: 0.9987	Epoch 12/20 Train Loss: 0.0714 Acc: 0.9864 Val Loss: 0.0461 Acc: 0.9871
Epoch 13/20 Train Loss: 0.0004 Acc: 1.0000 Val Loss: 0.0061 Acc: 0.9987	Epoch 13/20 Train Loss: 0.0884 Acc: 0.9835 Val Loss: 0.0506 Acc: 0.9935
Epoch 14/20 Train Loss: 0.0003 Acc: 1.0000 Val Loss: 0.0059 Acc: 0.9987	Epoch 14/20 Train Loss: 0.0515 Acc: 0.9916 Val Loss: 0.0358 Acc: 0.9948
Epoch 15/20 Train Loss: 0.0002 Acc: 1.0000 Val Loss: 0.0058 Acc: 0.9987	Epoch 15/20 Train Loss: 0.1101 Acc: 0.9780 Val Loss: 0.2219 Acc: 0.9612
Epoch 16/20 Train Loss: 0.0002 Acc: 1.0000 Val Loss: 0.0056 Acc: 0.9987	Epoch 16/20 Train Loss: 0.1986 Acc: 0.9541 Val Loss: 0.0627 Acc: 0.9910
Epoch 17/20 Train Loss: 0.0002 Acc: 1.0000 Val Loss: 0.0057 Acc: 0.9987	Epoch 17/20 Train Loss: 0.0412 Acc: 0.9955 Val Loss: 0.0388 Acc: 0.9961
Epoch 18/20 Train Loss: 0.0001 Acc: 1.0000 Val Loss: 0.0055 Acc: 0.9987	Epoch 18/20 Train Loss: 0.0819 Acc: 0.9874 Val Loss: 0.1041 Acc: 0.9832
Epoch 19/20 Train Loss: 0.0001 Acc: 1.0000 Val Loss: 0.0055 Acc: 0.9987	Epoch 19/20 Train Loss: 0.0460 Acc: 0.9961 Val Loss: 0.0699 Acc: 0.9897
Epoch 20/20	Epoch 20/20

Train Loss: 0.0001 Acc: 1.0000 Val Loss: 0.0055 Acc: 0.9987	Train Loss: 0.0862 Acc: 0.9855 Val Loss: 0.0663 Acc: 0.9858
--	--

۶) مواردی که مدل پیش‌بینی اشتباه انجام می‌دهد:

- شباهت ظاهری زیاد بین کلاس‌ها (Inter-class similarity): برای مثال در دیتاست راننده، رانندگی عادی با صحبت با تلفن و دست روی صورت ممکن است اشتباه شود، اگر زاویه دوربین طوری باشد که گوشی دیده نشود، مدل ممکنه است کلاس "normal driving" را پیش‌بینی کند به جای آنکه "talking on phone" انتخاب شود. دلیل این اتفاق این است که ViT تصویر را به patch‌های مستقل تقسیم می‌کند و تمرکز global دارد، اما اگر ویژگی کلیدی (مثلاً گوشی) در چند patch کوچک باشد، ممکن است attention کافی دریافت نکند
- وقتی شیء مهم خیلی کوچک باشد. برای مثال اگر گوشی فقط ۵٪ تصویر را اشغال کند و Patch size = 16 ممکن است گوشی داخل یک patch کوچک گم شود. ViT بر خلاف CNN حساسیت ذاتی به ویژگی‌های محلی ندارد. کانولوشن‌های لایه اول CNN معمولاً بهتر edge و object کوچک را می‌گیرند
- تغییر شدید نور یا سایه: برای مثال وقتی تصویر داخل تونل است نور از یک سمت شدید است در چنین نمونه‌هایی اگر train کم بوده باشند مدل در inference دچار خطا می‌شود. زیرا Transformer به distribution داده حساس است. اگر domain shift اتفاق بیفتد آنگاه دقت افت می‌کند.
- پوشانده شدن: برای مثال دست جلوی صورت یا فرمان بخشی از بدن را پوشانده باشد در این صورت چون attention روی کل تصویر پخش می‌شود، گاهی مدل روی نواحی غیرمرتبط تمرکز می‌کند. که میتوان این را با attention map دید.
- کلاس‌های نامتوازن (Class Imbalance): در این حالت برای مثال اگر کلاس ۰ شامل ۵۰۰۰ تصویر و کلاس ۷ شامل ۳۰۰ تصویر باشد، مدل تمایل دارد کلاس پرنمونه را بیشتر پیش‌بینی کند.
- Overfitting: مدل روی داده‌های آموزشی pattern حفظ کرده ولی generalize نکرده. این اتفاق در حالتی رخ میدهد که پارامتر زیاد باشد ولی دیتاست محدود داشته باشیم.

- پس زمینه گمراه کننده (Background Bias): در این حالت اگر در دیتای آموزشی، بیشتر تصاویر **texting** داخل ماشین سفید باشد مدل ممکن است رنگ داخلی ماشین را به عنوان **feature** استفاده کند. در نتیجه ممکن است در ماشین مشکی اشتباه کند.
- حالت های بینابینی (Ambiguous cases): در این حالت ممکن است راننده در حال خمیازه کشیدن یا در حال صحبت باشد یا دست نزدیک صورت باشد اما گوشی نداشته باشد. مدل **ambiguity** را نمی فهمد. فقط الگوهای آماری را می بیند.

برای تحلیل این خطاها می توانیم از **Confusion Matrix** استفاده کنیم. در این صورت می فهمیم کدام کلاس ها بیشتر اشتباه می شوند. همچنین می توانیم از **Attention Map Visualization** بفهمیم مدل به کجا نگاه می کند.

مدل **Vision Transformer** آموزش دیده در شرایطی که ویژگی های تشخیصی (قسمت های مهم تصویر) کوچک، پنهان یا از نظر بصری شبیه به هم باشند، با مشکل مواجه می شود. به دلیل عدم وجود سوگیری محلی قوی (**inductive bias**) در مقایسه با شبکه های **CNN**، مدل ممکن است در تشخیص جزئیات محلی ریز دقت کافی نداشته باشد. علاوه بر این، عدم تعادل کلاس ها و تغییرات توزیع داده (مثلاً تغییرات نور) عملکرد را بیشتر کاهش می دهد. تحلیل خطا با استفاده از ماتریس درهم ریختگی (**confusion matrix**) نشان داد که فعالیت های راننده که از نظر ظاهری شبیه هستند، اغلب اشتباه طبقه بندی می شوند.

۷) در این بخش به بررسی مفهوم **Temporal Attention** و تفاوت آن با رویکرد **Spatial Attention** که در مدل پیاده سازی شده فعلی استفاده کرده ام، می پردازیم. همچنین دلایل عدم امکان به کارگیری این مکانیزم در پروژه حاضر با توجه به ماهیت مجموعه داده مورد استفاده تشریح خواهد شد.

مدل **Vision Transformer** پیاده سازی شده در این پروژه از مکانیزم **Spatial Attention** بهره می برد. در این رویکرد، تصویر ورودی به پچ های کوچک تقسیم شده و مکانیزم توجه، ارتباط بین این پچ ها را در همان تصویر ثابت برقرار می کند. به عبارت دیگر، مدل یاد می گیرد که در یک لحظه مشخص، کدام نواحی تصویر برای تشخیص وضعیت راننده اهمیت بیشتری دارند. برای مثال، در تشخیص فعالیت "نوشیدن آب"، مدل با استفاده از **Spatial Attention** به نواحی شامل دست ها و لیوان توجه می کند و بر اساس همین اطلاعات مکانی، تصمیم گیری نهایی را انجام می دهد.

در مقابل، Temporal Attention مکانیزمی است که برای پردازش داده‌های دنباله‌ای مانند ویدیو طراحی شده است. در این رویکرد، مدل علاوه بر درک اهمیت مکانی هر ناحیه در یک فریم، قادر به برقراری ارتباط بین فریم‌های متوالی نیز می‌باشد. به این ترتیب، مدل نه تنها متوجه می‌شود که در یک لحظه خاص چه اتفاقی در حال رخ دادن است، بلکه می‌تواند توالی رویدادها و الگوهای رفتاری را نیز درک کند. برای نمونه، در تشخیص خواب‌آلودگی راننده، Temporal Attention می‌تواند توالی فریم‌هایی شامل پلک زدن مکرر، خمیازه کشیدن و نهایتاً بسته شدن چشم‌ها را به عنوان یک الگوی زمانی تشخیص داده و هشدار مناسب صادر کند. همچنین در فعالیت‌های پیچیده‌تر مانند صحبت با تلفن همراه، این مکانیزم قادر است حرکت دست از فرمان به سمت گوشی و قرار گرفتن آن در کنار گوش را به عنوان یک توالی معنادار شناسایی نماید.

با وجود مزایای قابل توجه Temporal Attention در تحلیل رفتار راننده در طول زمان، استفاده از این مکانیزم در پروژه حاضر امکان‌پذیر نبوده است. دلیل اصلی این محدودیت، ماهیت مجموعه داده مورد استفاده می‌باشد. دیتاست به کار گرفته شده در این پروژه از نوع تصاویر ثابت است و هر نمونه شامل یک فریم مجزا از راننده در یکی از ده حالت تعریف شده می‌باشد. به عبارت دیگر، داده‌ها فاقد بعد زمان بوده و دنباله‌ای از فریم‌های متوالی برای تحلیل رفتار در طول زمان در اختیار نمی‌باشد. از آنجایی که مکانیزم Temporal Attention برای پردازش ویدیو و توالی فریم‌ها طراحی شده است، اعمال آن بر روی مجموعه داده تصاویر ثابت عملاً امکان‌پذیر نبوده و نیازمند داده‌های ویدیویی با فریم‌های متوالی و برچسب‌گذاری زمانی مناسب می‌باشد.

در مجموع، می‌توان نتیجه گرفت که اگرچه استفاده از Temporal Attention می‌تواند بهبود قابل توجهی در تشخیص وضعیت‌های راننده ایجاد کند، اما پیاده‌سازی آن مستلزم در اختیار داشتن مجموعه داده ویدیویی مناسب می‌باشد. با توجه به محدودیت‌های موجود در پروژه حاضر، مدل طراحی شده بر اساس Spatial Attention بر روی تصاویر ثابت، عملکرد قابل قبولی در تشخیص ده حالت مختلف راننده از خود نشان داده است. برای پروژه‌های آتی که امکان جمع‌آوری داده‌های ویدیویی فراهم باشد، ترکیب هر دو مکانیزم Spatial و Temporal در معماری‌هایی نظیر TimeSformer یا ViViT می‌تواند منجر به ایجاد سامانه‌ای با دقت و قابلیت اطمینان بالاتر در پایش وضعیت راننده گردد.

۸) یکی از الزامات مهم در طراحی سامانه پایش وضعیت راننده، سرعت بالای استنتاج است. این سامانه باید بتواند در زمان واقعی (real-time) وضعیت راننده را تحلیل کرده و در صورت تشخیص موقعیت خطرناک،

هشدار لازم را صادر کند. از این رو، بهینه‌سازی سرعت استنتاج بدون کاهش قابل توجه دقت، از اهمیت ویژه‌ای برخوردار است. در این پروژه، روش **Automatic Mixed Precision (AMP)** برای افزایش سرعت استنتاج مدل **Vision Transformer** انتخاب و پیاده‌سازی شده است.

مفهوم AMP

Automatic Mixed Precision یک تکنیک بهینه‌سازی است که در آن از ترکیب دو نوع داده با دقت متفاوت برای محاسبات استفاده می‌شود. در این روش، بیشتر عملیات محاسباتی با دقت **float16** (۱۶ بیت) انجام می‌شود که سرعت بالاتر و مصرف حافظه کمتری دارد، در حالی که عملیات حساس برای حفظ دقت نهایی مدل، همچنان با دقت **float32** (۳۲ بیت) محاسبه می‌گردند.

این روش برای اولین بار توسط **NVIDIA** معرفی شد و امروزه در اکثر فریمورک‌های یادگیری عمیق از جمله **PyTorch** به صورت خودکار پشتیبانی می‌شود.

دلایل انتخاب AMP

روش **AMP** به دلایل متعددی به عنوان گزینه اصلی برای افزایش سرعت استنتاج در این پروژه انتخاب شده است:

اول، عدم تغییر در معماری مدل. برخلاف روش‌هایی مانند کاهش ابعاد **embedding**، افزایش اندازه پیچ یا هرس کردن (**pruning**) که نیازمند تغییر در ساختار مدل و گاهی آموزش مجدد هستند، **AMP** بدون کوچک‌ترین تغییری در معماری مدل اعمال می‌شود. این ویژگی باعث می‌شود مدل طراحی شده دقیقاً با همان ساختار اولیه خود باقی بماند و تنها نحوه محاسبات بهینه‌سازی شود.

دوم، حفظ دقت مدل. در **AMP**، عملیات حساس مانند نرمال‌سازی، **softmax** و تجمیع گرادینت‌ها همچنان با دقت **float32** انجام می‌شوند. این امر تضمین می‌کند که دقت نهایی مدل کاهش قابل توجهی پیدا نمی‌کند و عملکرد آن در تشخیص وضعیت‌های راننده تقریباً در همان سطح قبلی باقی می‌ماند.

سوم، بهره‌گیری از قابلیت‌های سخت‌افزاری **GPU** های مدرن از جمله سری **RTX** شرکت **NVIDIA** دارای واحدهای سخت‌افزاری ویژه‌ای به نام **Tensor Cores** هستند که برای محاسبات با دقت مختلط طراحی شده‌اند. **AMP** این هسته‌ها را فعال کرده و می‌تواند سرعت استنتاج را تا دو برابر افزایش دهد.

چهارم، کاهش مصرف حافظه. استفاده از **float16** به جای **float32**، حجم داده‌ها را به نصف کاهش می‌دهد. این کاهش حجم، پهنای باند حافظه را آزاد کرده و امکان پردازش همزمان داده‌های بیشتر را فراهم می‌کند.

پنجم، پیاده‌سازی ساده AMP در PyTorch با افزودن تنها چند خط کد قابل پیاده‌سازی است و نیازی به تغییرات گسترده در ساختار پروژه ندارد.

مقایسه با سایر روش‌ها

روش‌های متعددی برای افزایش سرعت استنتاج وجود دارند که هر یک مزایا و محدودیت‌های خاص خود را دارند:

کاهش ابعاد embedding یا تعداد لایه‌ها می‌تواند سرعت را افزایش دهد، اما این کار مستلزم تغییر معماری مدل و گاهی آموزش مجدد است و معمولاً با کاهش دقت همراه می‌باشد.

افزایش اندازه پیچ، تعداد توکن‌های ورودی به مدل را کاهش می‌دهد و محاسبات را سبک‌تر می‌کند، اما جزئیات فضایی تصویر را از بین می‌برد و می‌تواند بر دقت تشخیص تأثیر منفی بگذارد.

Quantization یا کاهش دقت داده به ۸ بیت، سرعت را بسیار افزایش می‌دهد اما نیازمند کالیبراسیون دقیق و گاهی باعث افت محسوس دقت می‌شود.

هرس کردن (pruning) نیز وزن‌های کم‌اهمیت را حذف می‌کند، اما برای دستیابی به نتیجه مطلوب نیاز به آموزش مجدد و تنظیم دقیق دارد.

در مقابل، AMP بدون نیاز به تغییر معماری، آموزش مجدد یا کاهش قابل توجه دقت، افزایش سرعت مناسبی ارائه می‌دهد.